

# Database Architecture

# Prev

- Data
- Database
- DBMS
- DBMS Tools
- Advantages
- When Use or Not

# Type Architecture in DBMS

## 1-Tier Architecture

In 1-Tier Architecture the database is directly available to the user, the user can directly sit on the DBMS and use it that is, the client, server, and Database are all present on the same machine.

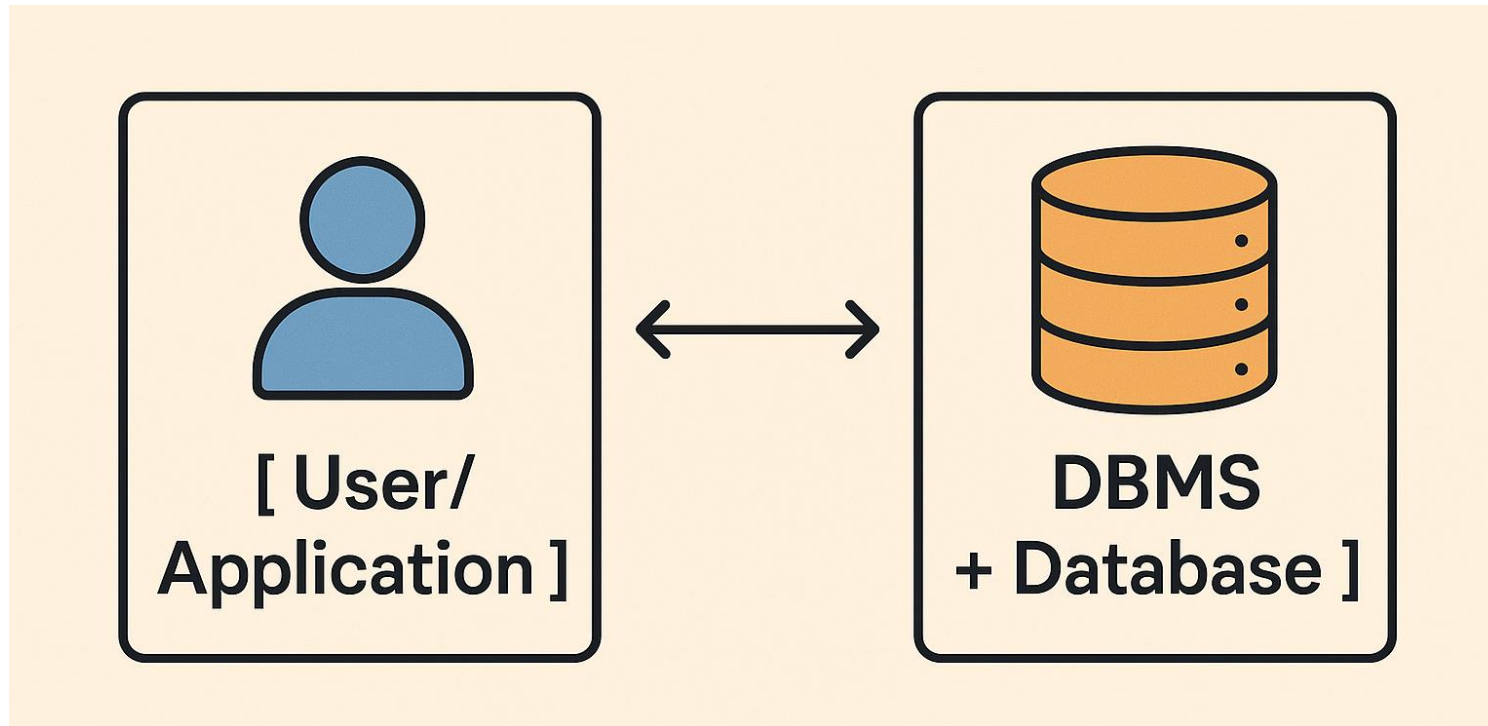
## 2-Tier Architecture

The 2-tier architecture is similar to a basic. The application at the client end directly communicates with the database on the server side.

## 3-Tier Architecture

In DBMS, the 3-tier architecture is a client-server architecture that separates the user interface, application processing, and data management into three distinct tiers or layers. The 3-tier architecture is widely used in modern web applications and enterprise systems because it offers scalability, flexibility, and security.

# 1-Tier Architecture in DBMS



- The user interacts directly with the database, without a middle layer.
- The DBMS and the application both reside on the same system.

# 1-Tier Architecture in DBMS (2)

## Advantages

- Simple to design and implement.
- Fast performance since there is no network overhead.
- Suitable for personal use or small-scale applications.

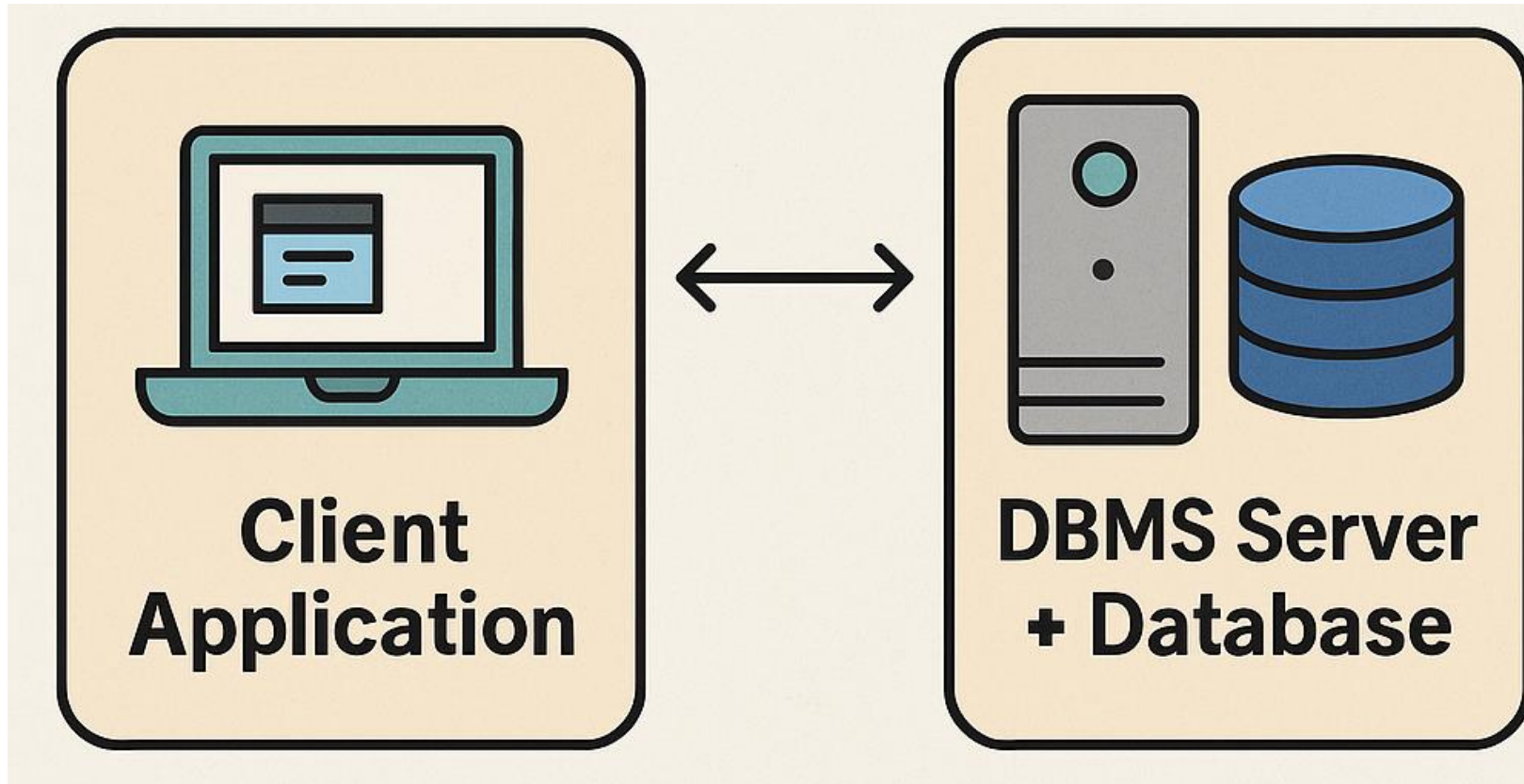
## Disadvantages

- Poor scalability and security.
- Direct user access increases the risk of accidental or malicious data manipulation.
- Not practical for multi-user or distributed systems.

## Sample Case

Microsoft Access used on a single PC for personal data management.  
Standalone desktop applications like SQLite for small local apps.

# 2-Tier Architecture in DBMS



- Client and server model.
- The client (application) directly communicates with the DBMS server.
- Business logic may be partly on the client side.

# 2-Tier Architecture in DBMS (2)

## Advantages

- Faster than 3-tier for small applications since there's no middleware.
- Easier to maintain than 1-tier due to separation of client and server.
- Supports multiple users.

## Disadvantages

- Limited scalability.
- Clients are tightly coupled with the DBMS.
- Security concerns, as clients may still have direct access to the database.

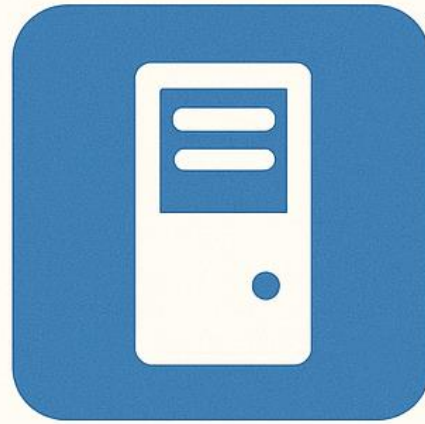
## Sample

- Banking system desktop applications where clients connect directly to the database.
- Educational institution management systems with a shared database server.

# 3-Tier Architecture in DBMS



[ Client (UI) ]



[ Application Server  
(Business Logic) ]



[ DBMS Server  
+ Database ]

Divides the system into three layers:

**1. Presentation Layer** (UI)

**2. Application Layer** (business logic / middleware)

**3. Database Layer** (DBMS and database)

# 3-Tier Architecture in DBMS (2)

## Advantages

- High scalability, flexibility, and security.
- Business logic is centralized in the application layer, reducing client-side load.
- Easier to maintain and update (changes in one layer don't affect others much).

## Disadvantages

- More complex and costly to implement.
- Slightly slower due to middleware overhead.
- Requires skilled setup and maintenance.

## Sample Case

- Web-based applications (e.g., e-commerce websites, online banking portals).
- Enterprise-level systems (ERP, CRM).
- Applications using frameworks like Java EE, .NET, or Spring Boot.

# Schema Architecture in DBMS

**Schema Architecture** hides the details of DB from the user. Its other name is Three Levels of Abstraction. The database administrator should be able to change the structure of the database according to need without affecting the user's view. This effective three schema architecture holds 3-layers as below.

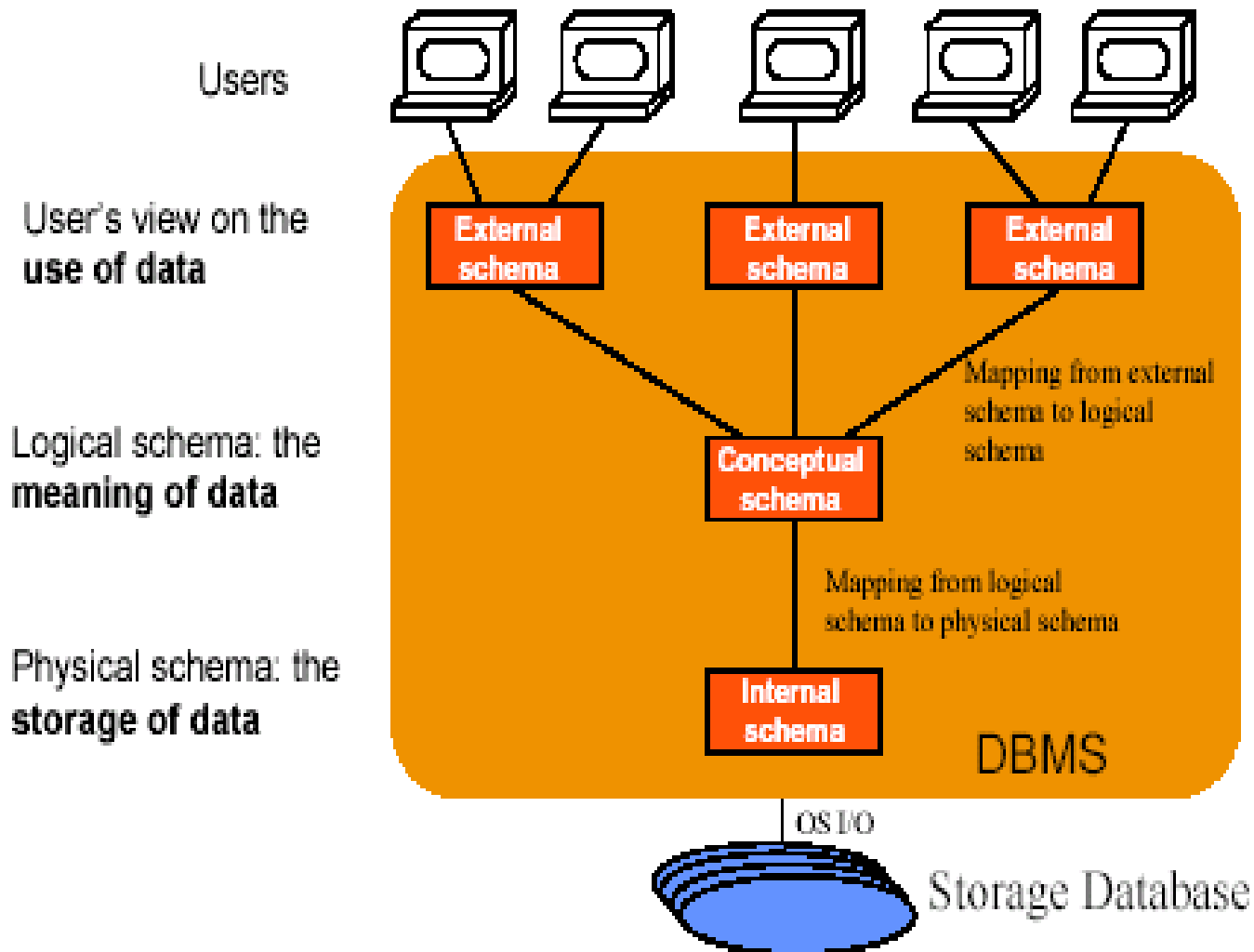
1. Internal level (administrator, physical level)
2. Conceptual level (database designer, logical level)
3. External level (front-end developer, user level)

# Next Lesson

## “SCHEMA ARCHITECTURE DATABASES”



# Schema Architecture in DBMS



**1. Internal Schema**

**2. Conceptual Schema**

**3. External Schema**

# Internal Schema

## Definition

- The **lowest level of abstraction** in a DBMS.
- Describes *how the data is physically stored* (e.g., indexes, storage structure, compression, hashing).
- Concerned with **performance and efficiency**.

## Output

- Defines file organization, indexing, record placement, and access paths.
- Output is the **physical data structure**.

```
-- Creating an index (internal schema detail)
CREATE INDEX idx_customer_name
ON Customers (CustomerName);
```

# Conceptual Schema

## Definition

- The **logical level** of DBMS.
- Describes the **overall structure of the database**: entities, relationships, constraints.
- Independent of physical storage.

## Output

- Defines tables, attributes, primary/foreign keys, relationships.
- Output is the **logical data model**.

# Conceptual Schema (2)

*-- Defining Logical structure of data*

**CREATE TABLE** Customers (

CustomerID **INT PRIMARY KEY**,

CustomerName **NVARCHAR(100)**,

Email **NVARCHAR(100)**

);

**CREATE TABLE** Orders (

OrderID **INT PRIMARY KEY**,

CustomerID **INT**,

OrderDate **DATE**,

**FOREIGN KEY** (CustomerID) **REFERENCES** Customers(CustomerID)

);

# External Schema

## Definition

- The **highest level** (user view).
- Defines *how different users see the data* (different views for different needs).
- Provides **security** by restricting access to certain columns/rows.

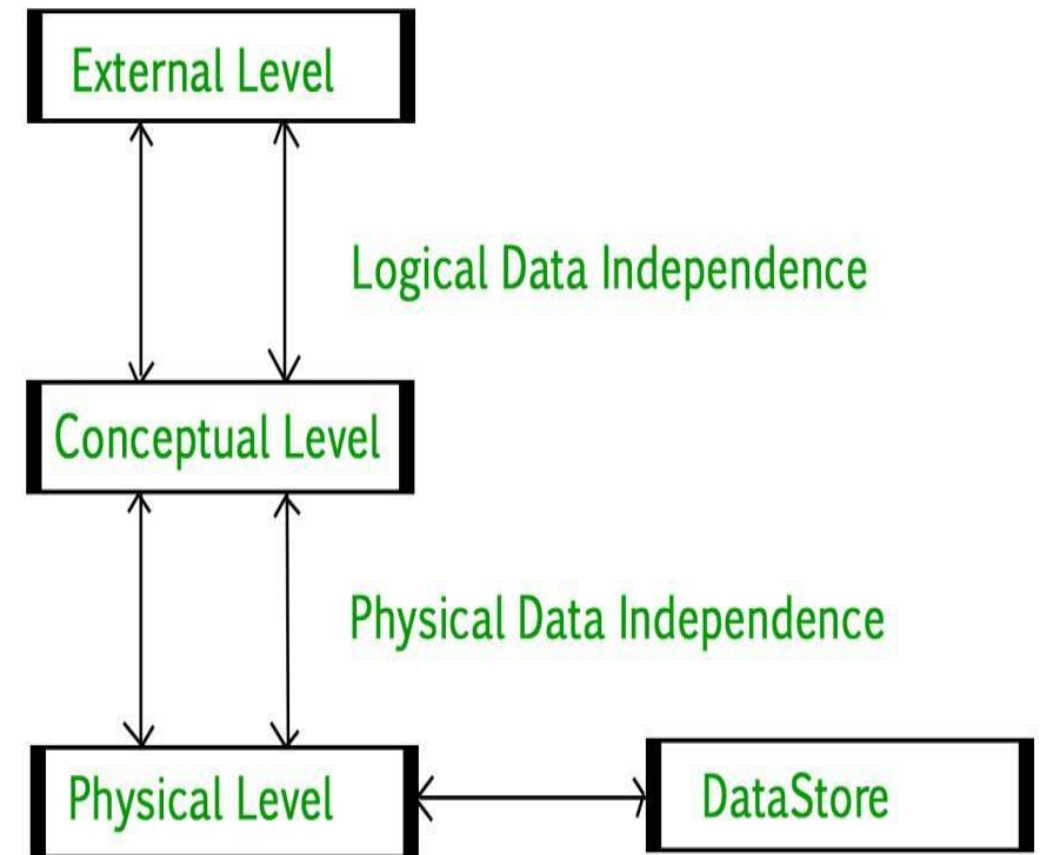
## Output

- Customized **user views** of the database.
- Different external schemas can exist for different users.

```
-- External schema: user-specific view  
  
CREATE VIEW CustomerOrders AS  
  
SELECT c.CustomerName, o.OrderID, o.OrderDate  
  
FROM Customers c  
  
JOIN Orders o ON c.CustomerID = o.CustomerID;
```

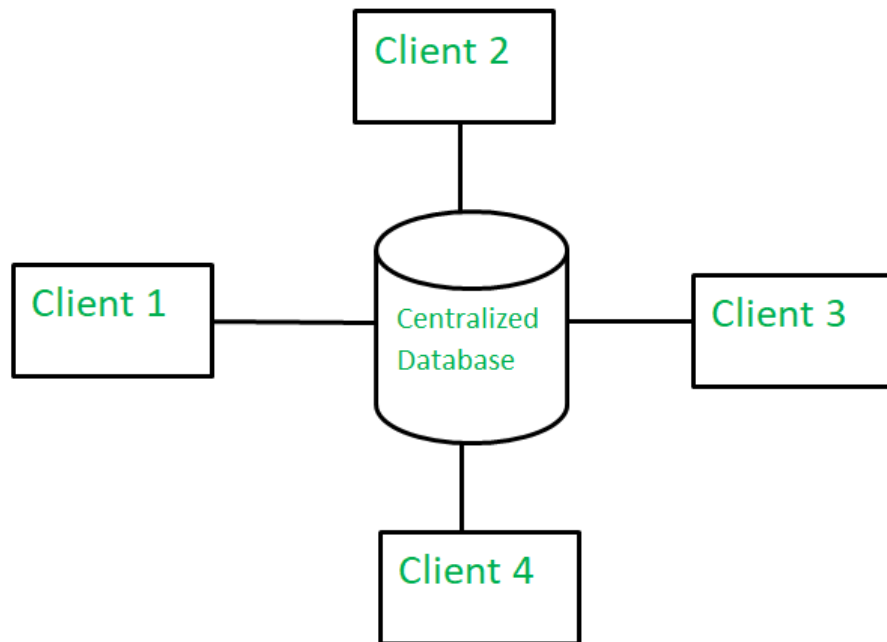
# Data Layer in DBMS

- **Logical Data Independence:** The capacity to change the conceptual schema without having to change the external schema and their application programs.
- **Physical Data Independence:** The capacity to change the internal schema without having to change the conceptual schema.

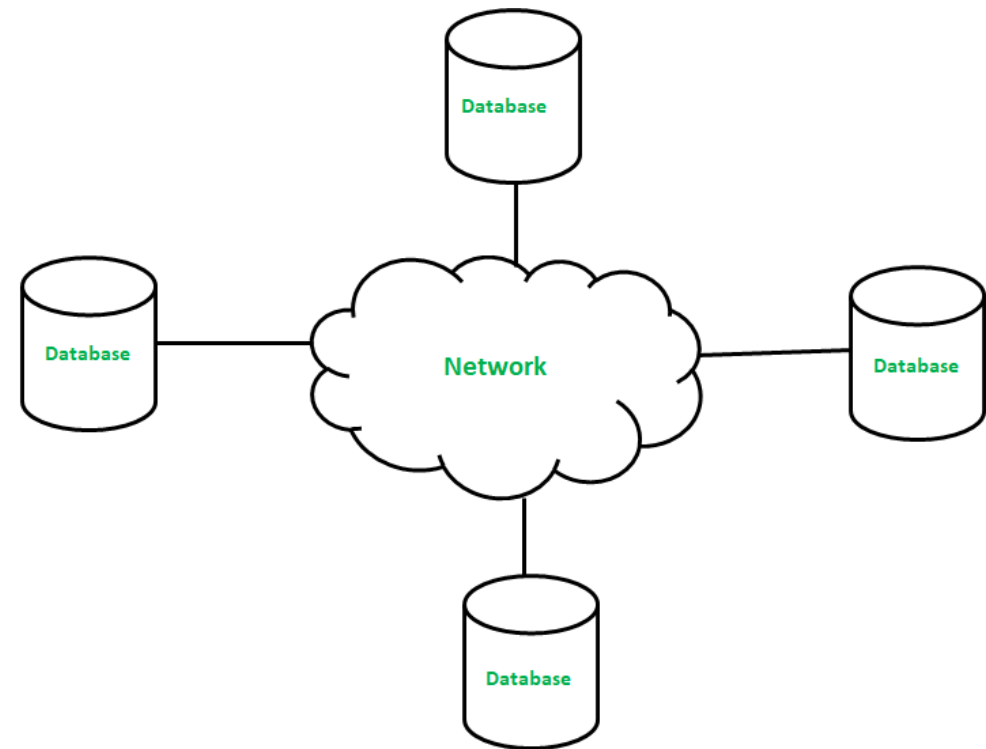


# Centralized Database vs Distributed Database

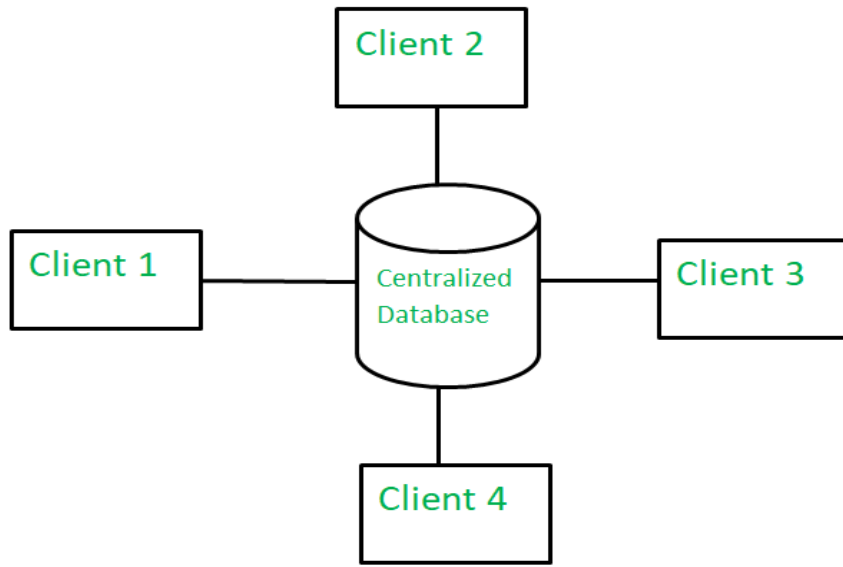
## Centralized Database



## Distributed Database



# Centralized Database



## A. Centralized Database:

A centralized database is basically a type of database that is stored, located as well as maintained at a single location only.

### Advantages:

- Since all data is stored at a single location only thus it is easier to access and coordinate data.
- The centralized database has very minimal data redundancy since all data is stored in a single place.
- It is cheaper in comparison to all other databases available.

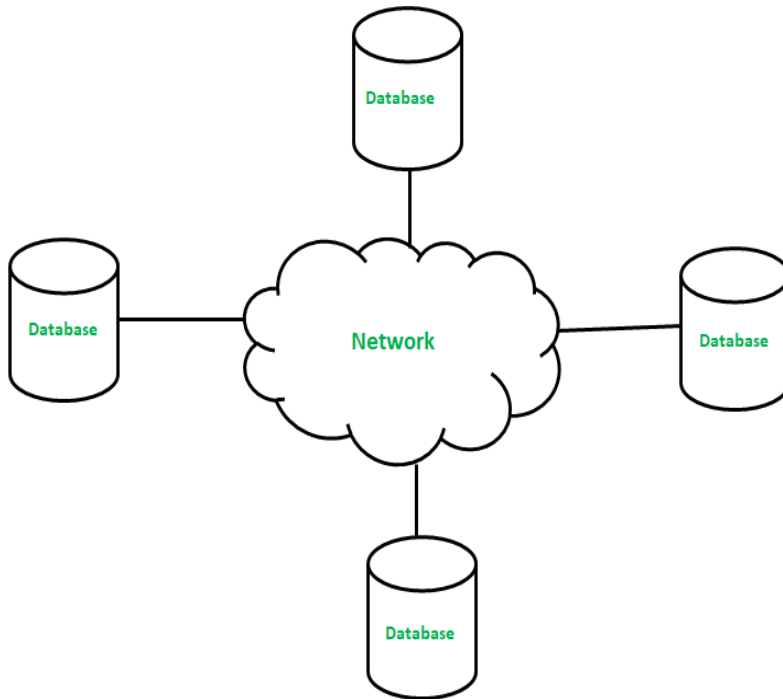
### Disadvantages:

- The data traffic in the case of a centralized database is more.
- If any kind of system failure occurs in the centralized system then the entire data will be destroyed.

# Distributed Database

## B. Distributed Database:

A distributed database is basically a type of database which consists of multiple databases that are connected with each other and are spread across different physical locations.



### Advantages:

- This database can be easily expanded as data is already spread across different physical locations.
- The distributed database can easily be accessed from different networks.
- This database is more secure in comparison to a centralized database.

### Disadvantages:

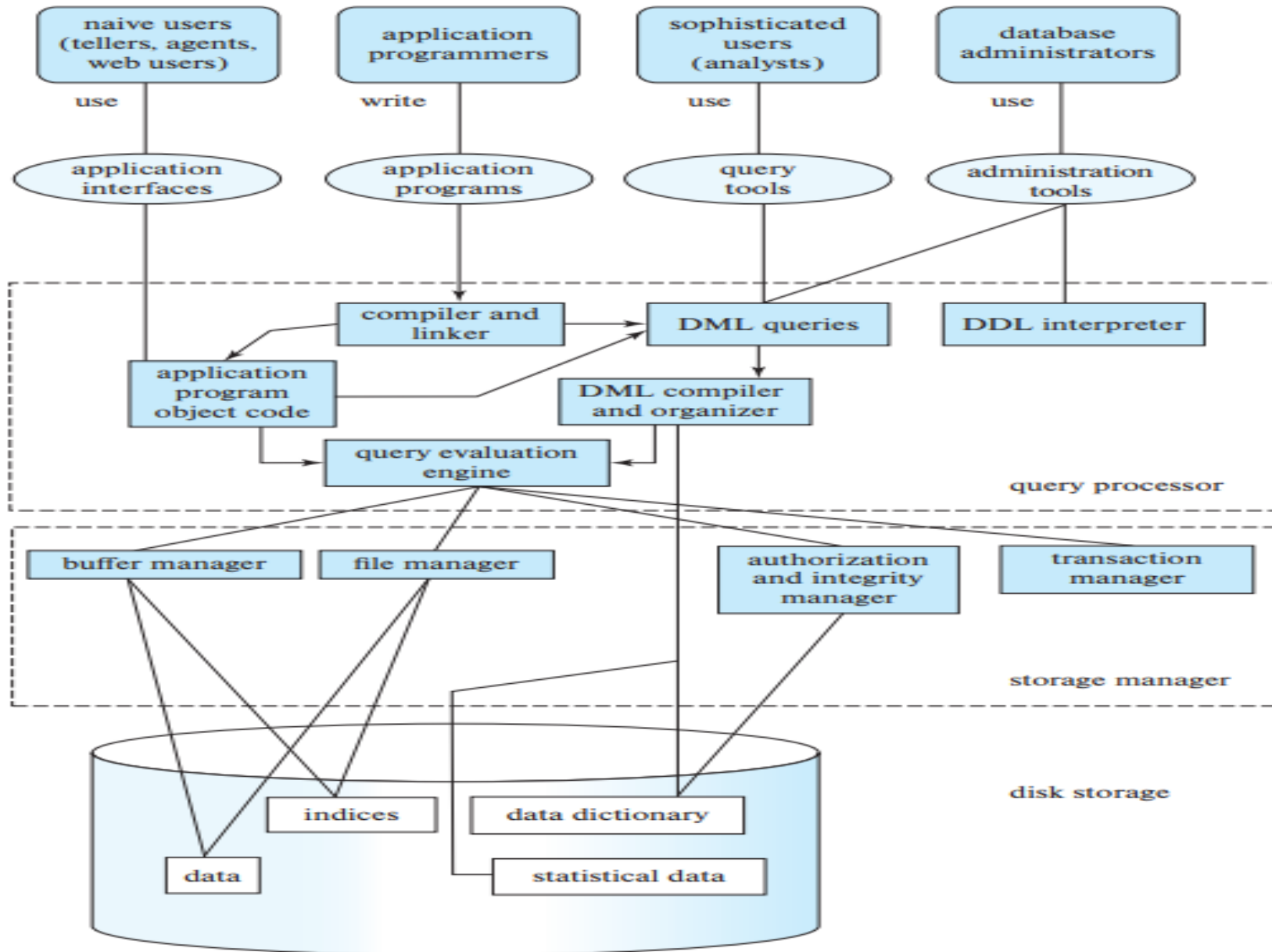
- This database is very costly and is difficult to maintain because of its complexity.
- In this database, it is difficult to provide a uniform view to users since it is spread across different physical locations.

**Sample: Almost E-commerce Platform**

# Variations of Distributed Environments:

- Homogeneous DBMS
  - Same engine
- Heterogeneous DBMS
  - Different engine
- Federated or Multi-database Systems
  - Unite multi-dbms

# Database App System Architecture

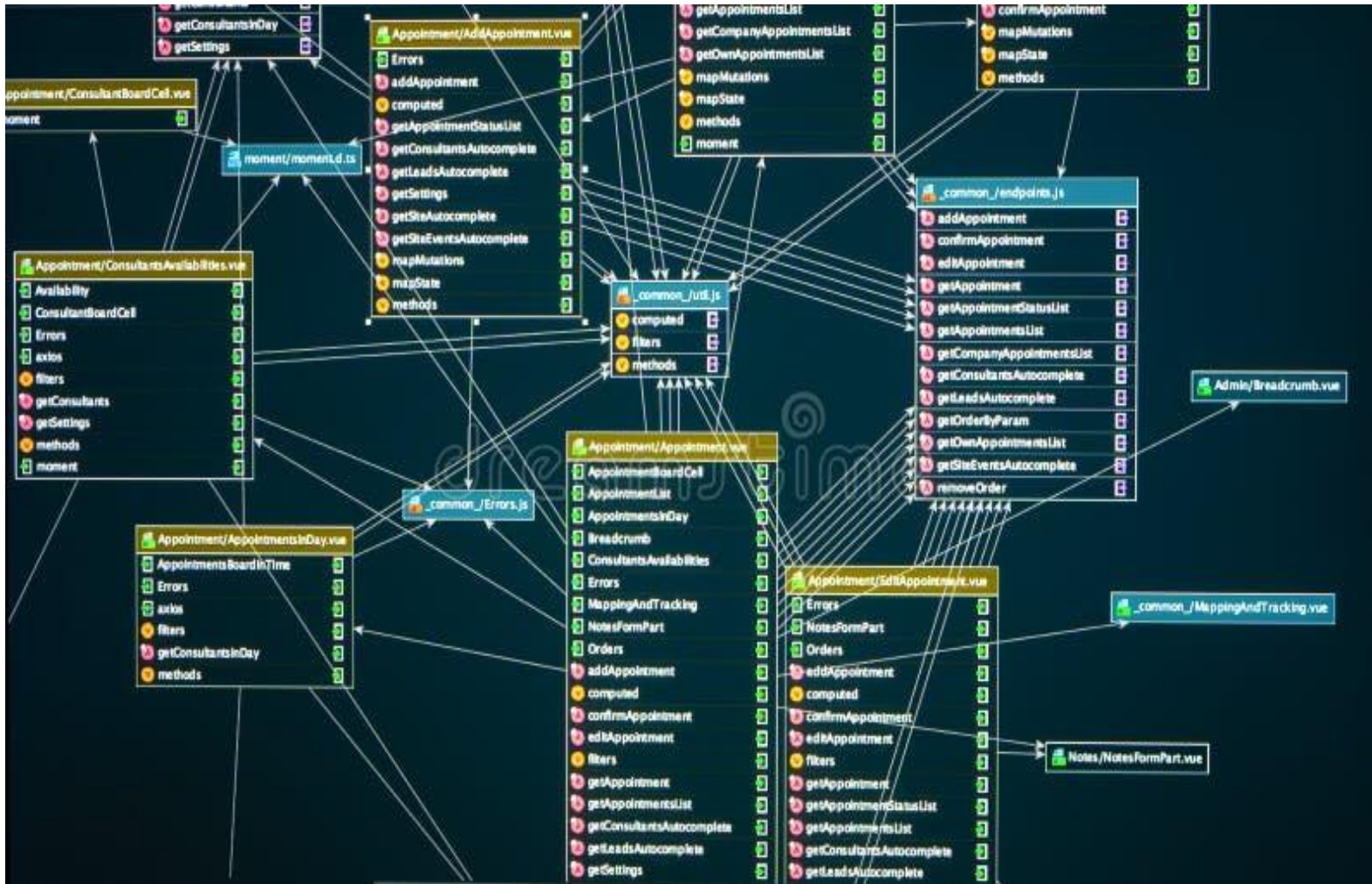


# Next Lesson

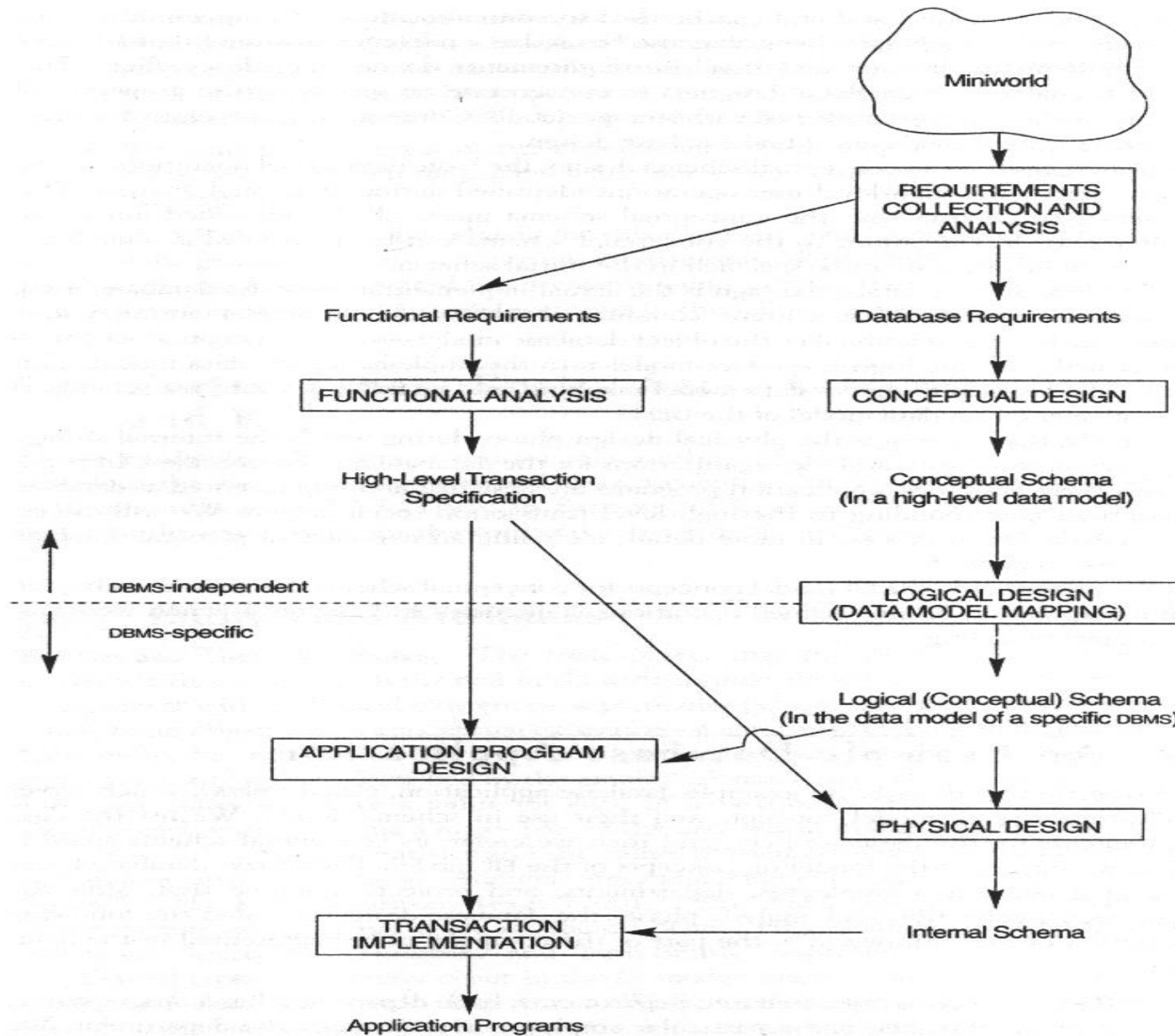
## “STRUCTURE RELATIONAL DATABASES”



# STRUCTURE RELATIONAL DATABASES



# Flow Design



# Conceptual Design

- ✓ High-level representation of the data.
- ✓ Focuses on **what data should be stored** and **relationships between data**.
- ✓ Independent of DBMS or hardware.

## Output

- **Entity-Relationship (ER) Diagram**

- Entities, attributes, and relationships, but no details on keys, data types, or indexes.

# Logical Design

- ✓ Converts conceptual schema into a **DBMS-independent logical model**.
- ✓ Defines **tables, primary keys, foreign keys, and normalization**.
- ✓ Still independent of the actual DBMS platform (SQL Server, Oracle, MySQL, etc.).

## Output

- Relational schema (normalized tables).
- Attributes with data types, keys, and relationships.

# Logical Design (2)

```
CREATE TABLE Customer (  
    CustomerID INT PRIMARY KEY,  
    Name NVARCHAR(100),  
    Contact NVARCHAR(50)  
);  
  
CREATE TABLE [Order] (  
    OrderID INT PRIMARY KEY,  
    CustomerID INT,  
    OrderDate DATE,  
    TotalAmount DECIMAL(10,2),  
    FOREIGN KEY (CustomerID) REFERENCES Customer(CustomerID)  
);
```

```
CREATE TABLE Product (  
    ProductID INT PRIMARY KEY,  
    Name NVARCHAR(100),  
    Price DECIMAL(10,2)  
);  
  
CREATE TABLE OrderProduct (  
    OrderID INT,  
    ProductID INT,  
    Quantity INT,  
    PRIMARY KEY (OrderID, ProductID),  
    FOREIGN KEY (OrderID) REFERENCES [Order](OrderID),  
    FOREIGN KEY (ProductID) REFERENCES Product(ProductID)  
);
```

# Physical Design

- ✓ Implementation of the logical design in a **specific DBMS (e.g., SQL Server)**.
- ✓ Focuses on **how the database is stored and optimized**.
- ✓ Includes indexing, partitions, clusters, storage paths, and performance tuning.

## Output

- Actual SQL DDL (Data Definition Language) scripts with indexes, constraints, storage.
- Considerations for performance, backups, and security.

# Physical Design

```
CREATE TABLE Customer (  
    CustomerID INT PRIMARY KEY CLUSTERED,  
    Name NVARCHAR(100) NOT NULL,  
    Contact NVARCHAR(50) NULL  
);  
  
CREATE TABLE [Order] (  
    OrderID INT PRIMARY KEY CLUSTERED,  
    CustomerID INT NOT NULL,  
    OrderDate DATE NOT NULL,  
    TotalAmount DECIMAL(10,2) NOT NULL,  
    FOREIGN KEY (CustomerID) REFERENCES Customer(CustomerID)  
);  
CREATE NONCLUSTERED INDEX idx_Order_CustomerID ON [Order](CustomerID)
```

# Step By Step Design ER Model

1. Define scope **Miniworld**
2. Data **needs analysis**
3. Define **entity**
4. Define strong/weak entity
5. Define attributes each entity
6. Define **Relationships**
7. Define **cardinality and constraints**

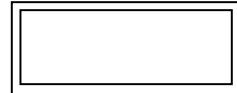
# NOTATION FOR ER SCHEMAS

Symbol

Meaning



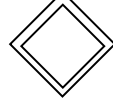
ENTITY TYPE



WEAK ENTITY TYPE



RELATIONSHIP TYPE



IDENTIFYING RELATIONSHIP TYPE



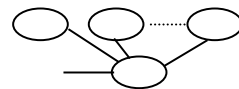
ATTRIBUTE



KEY ATTRIBUTE



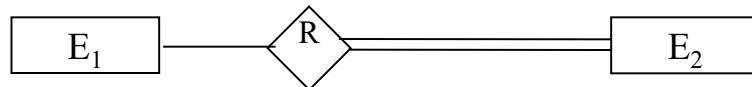
MULTIVALUED ATTRIBUTE



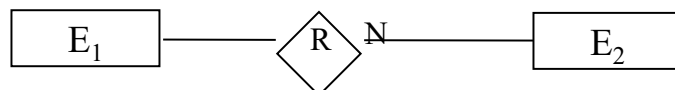
COMPOSITE ATTRIBUTE



DERIVED ATTRIBUTE



PARTICIPATION



CARDINALITY RATIO 1:N FOR  $E_1:E_2$  IN R



STRUCTURAL CONSTRAINT (min, max) ON PARTICIPATION OF E IN R

# Entity

Object/ a collection of information that is organized

## **Strong Entity**

Strong Entity is independent of any other entity in the schema

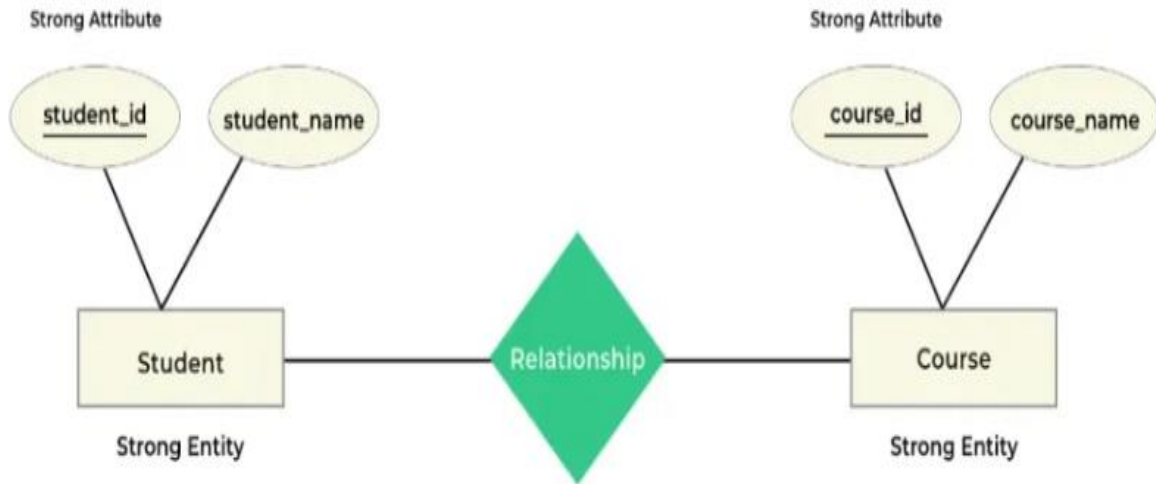
Example : A student entity can exist without needing any other entity in the schema  
or a course entity can exist without needing any other entity in the schema

## **Weak entity**

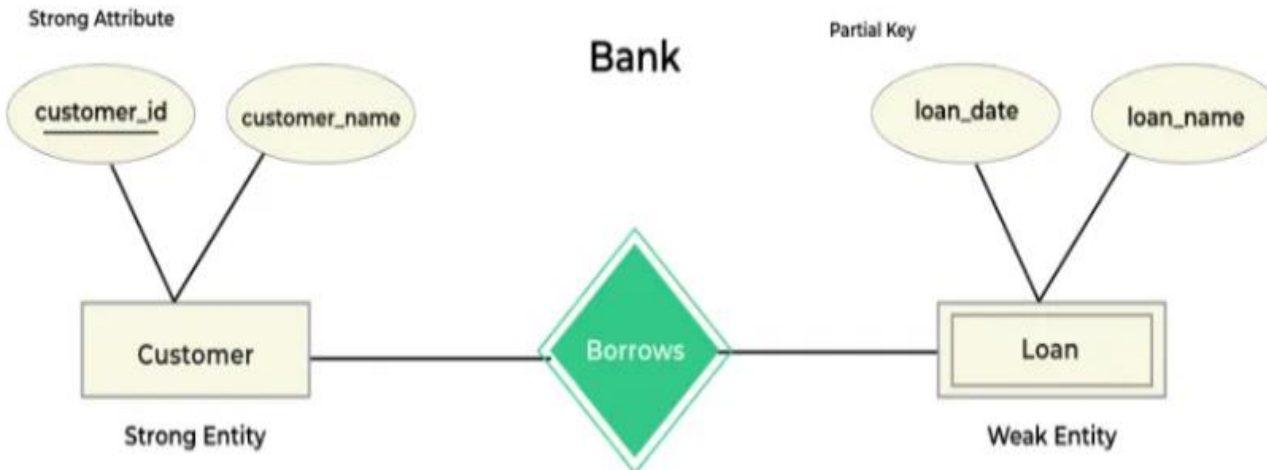
An entity that cannot be uniquely identified without a relationship to a strong entity.  
It depends on a strong entity's primary key plus its own partial key.

Example: Loan(CustomerID, LoanID, Ammount) → Each order item depends on an Order and a Product to exist.

# Entity (2)



**Strong**



**Weak**

# Attribute

Properties used to describe an entity

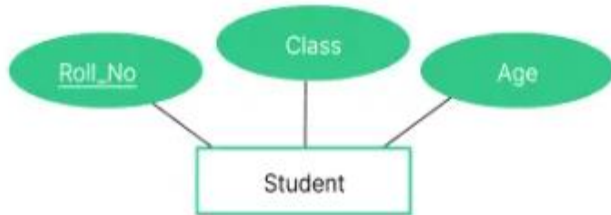
Attribute component information of entity

Types:

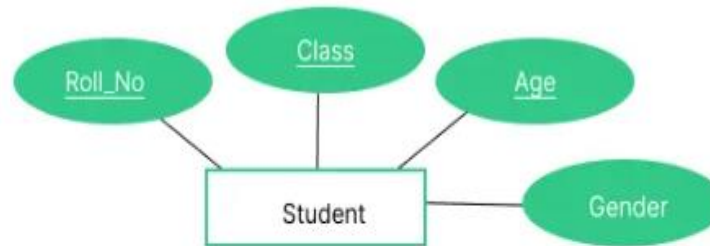
- a. **Simple.** Each entity has a single atomic value for the attribute.
- b. **Composite.** May be composed of several components.
- c. **Single valued** are those attributes which can take only one value for a given entity from an entity set.
- d. **Multi-valued.** May have multiple values for that attribute.
- e. **Derived.** Attribute which is calculated from a stored attribute
- f. **Key attributes** are those attributes which can identify an entity uniquely in an entity set.

# Attribute (2)

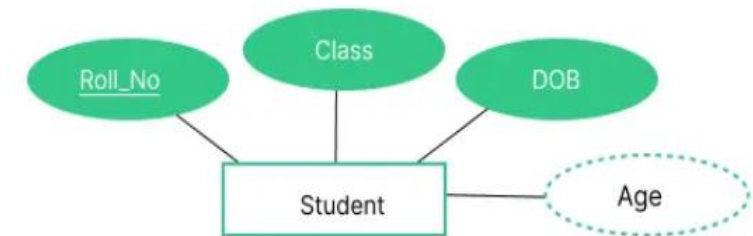
## Simple Attributes



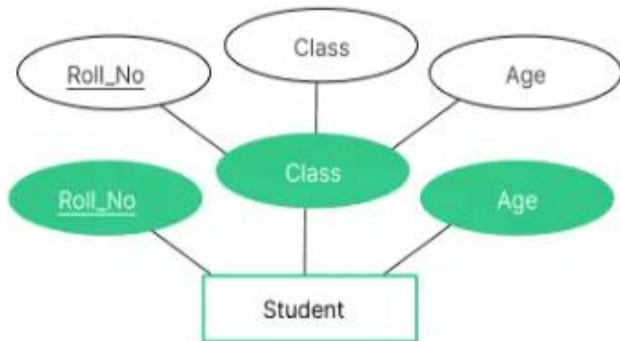
## Single Valued Attributes



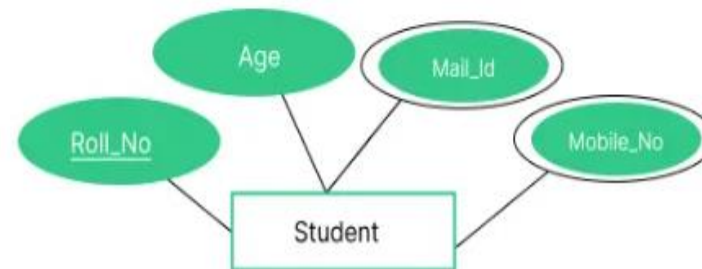
## Derived Attributes



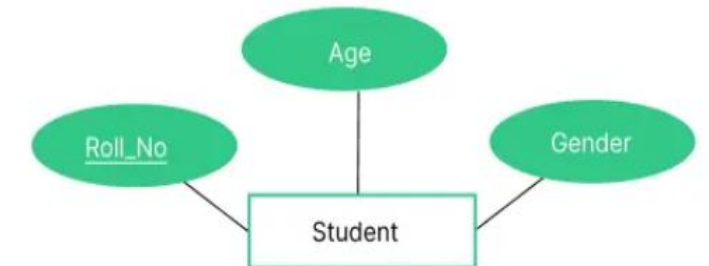
## Composite Attributes



## Multi Valued Attributes



## Key Attributes



# Attribute Value

As structured data, each attribute has value which is defined by a specific datatype. Possible values of an attribute

## **Datatype:**

Number. Integer, byte, Float

String. Char, Varchar, Text

Boolean..etc

## **Can be more specific, for example:**

Birth date must be > 1850 but before <(now)

Sex must be either female or male

**Note: each engine may have slight different datatype**

# Key Attribute

- ❖ What makes an entity unique?
- ❖ Selection of the keys is an important part of the database design
  - It affects integrity validation and performance
  - Declaring an attribute as a key, and declaring it 'duplicates not allowed' we can prevent users from entering erroneous duplicate data
  - The key can also maintain integrity by linking the key with a key in another table.

## Key Attribute (2)

- A key attribute may be composite.
  - For example, VehicleTagNumber is a key of the CAR entity type with components (Number, State).
- An entity type may have more than one key.
- For example, the CAR entity type may have two keys:
  - VehicleIdentificationNumber (popularly called VIN) and
  - VehicleTagNumber (Number, State), also known as license\_plate number.

(VehicleIdentificationNumber, VehicleTagNumber)

## Key Attribute (3)

- Key can be formed as artificial key
- When existing attributes are difficult to maintain the job as a key
  - Entity MAHASISWA has NIM as a key attribute L231901 , to make sure each of data is unique

# Relationship

- Relates two or more distinct entities with a specific meaning.
- Note: active word is preferred

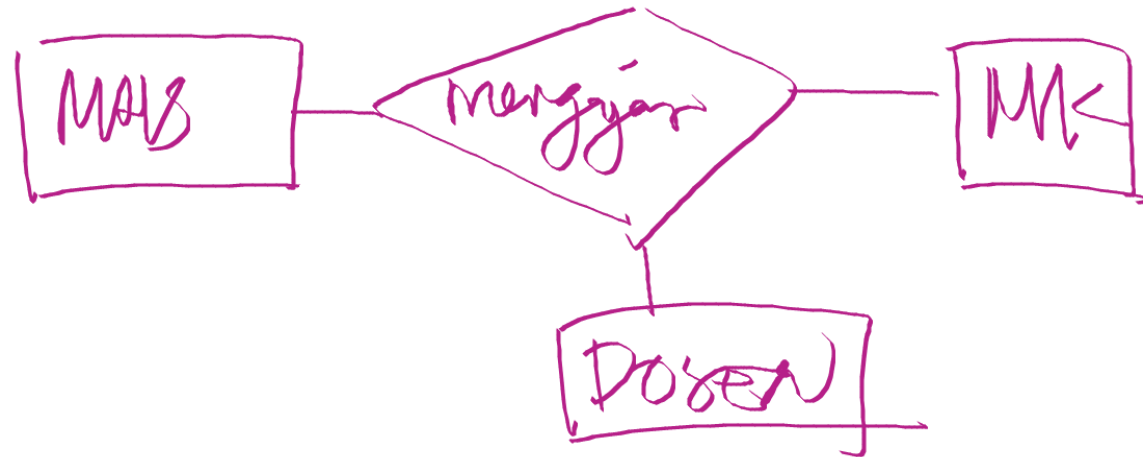
## **Variety:**

1. Binary. Among 2 entities
  - It is a normal form of relationship
2. Ternary. Relationship only existed if 3 entities are involved
3. N-Ary. Among N entities. Relationship only existed if N entities are involved

**Note: choose as like as what exist in miniworld**

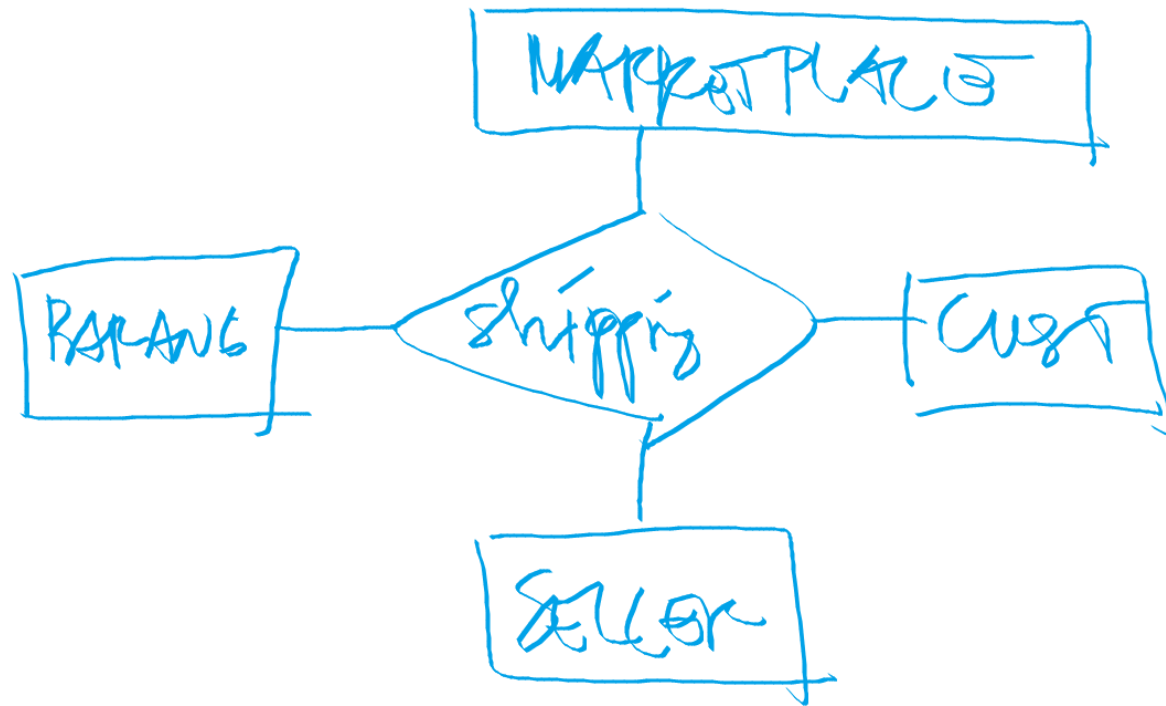
# Relationship (Ternary)

Mengajar



# Relationship (N-ary)

Shipping

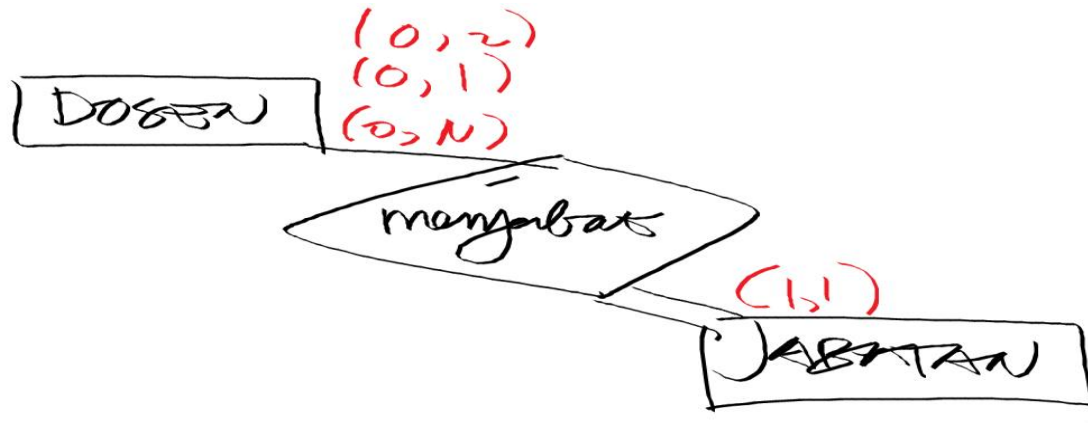


# Naming Convention

1. Choose names (for entity types, attributes, relationship types) that convey the meanings attached to the different constructs in the schema
2. Use **singular** names for entity types
3. Use **uppercase** for entity type and relation type names
4. Use **lowercase** for role names
5. **Nouns** tend to give rise to entity type names
6. **Verbs** tend to indicate names of relationship types

# Cardinality

- cardinality defines the number of instances of one entity that can be associated with instances of another entity. It is expressed using minimum and maximum cardinality values.
- (Min, Max)
  - How many min number of instance and max number of instance which can get involved in the relationship
  - Note: for partial participation (min must be 0), for total participation (min must be 1)



# Notation Cardinality

Cardinality and Participation for Popular Methodologies

| Method    | Cardinality | Participation | Example: An employee can work for one department; A department can have one or more employees |
|-----------|-------------|---------------|-----------------------------------------------------------------------------------------------|
| Chen      | Look-Across | Look-Here     |                                                                                               |
| Crowsfeet | Look Across | Look Across   |                                                                                               |
| I.E.      | Look Across | Look Across   |                                                                                               |
| Min/Max   | Look Here   | Look here     |                                                                                               |
| Min/Max   | Look Across | Look Across   |                                                                                               |



One



Many



One (and only one)



Zero or one



One or many



Zero or many

# Minimum Cardinality

The **smallest number** of entity instances that must participate in a relationship.

**Purpose:** Describes the *mandatory or optional* nature of participation.

## Example:

- In an Employee–Department relationship:
  - An employee must belong to **at least one department** (min cardinality = 1).
  - A department must have **at least zero employees** (min cardinality = 0, meaning it can be empty)

# Maximum Cardinality

The **largest number** of entity instances that can participate in a relationship.

- **Purpose:** Describes the *upper limit* of participation.

## Example:

- In a Student–Course relationship:
  - ✓ A student can enroll in **many courses** (max cardinality = many).
  - ✓ A course can have **many students** (max cardinality = many).
- So this would be a **many-to-many** relationship.

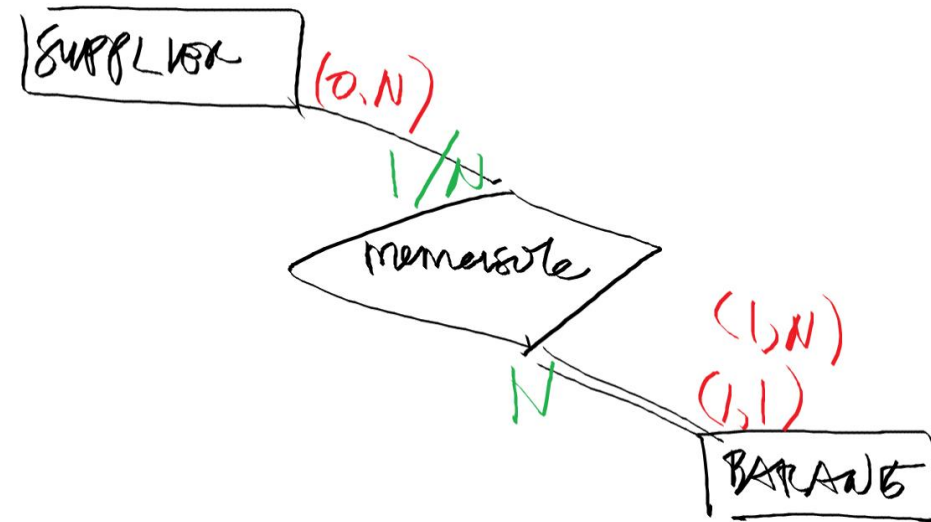
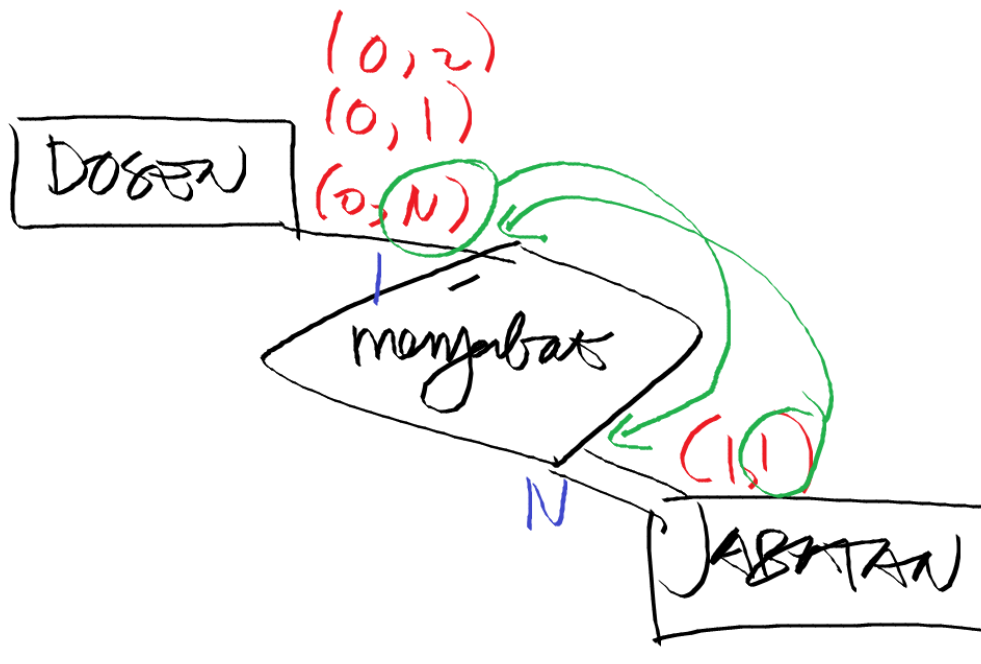
# Constraints on Relationships

## Constraints on Relationship Types

- ( Also known as ratio constraints )
- Maximum Cardinality
  1. One-to-one (1:1)
  2. One-to-many (1:N) or Many-to-one (N:1)
  3. Many-to-many
- Minimum Cardinality (also called participation constraint or existence dependency constraints)
  1. zero (optional participation, not existence-dependent)
  2. one or more (mandatory, existence-dependent)

# Constraints

- Maximum Cardinality



# Recursive Relationship

- We can also have a **recursive** relationship type.
- **Both participations are same entity type** in different roles.
- For example, SUPERVISION relationships between EMPLOYEE (in role of supervisor or boss) and (another) EMPLOYEE (in role of subordinate or worker).
- In following figure, first role participation labeled with 1 and second role participation labeled with 2.
- In ER diagram, need to display role names to distinguish participations.

- Dalam kasus marketplace
  - Akun dalam marketplace bisa sebagai penjual bisa sebagai pembeli



# Data Modeling Tools

| COMPANY                  | TOOL                                                            | FUNCTIONALITY                                                                |
|--------------------------|-----------------------------------------------------------------|------------------------------------------------------------------------------|
| Embarcadero Technologies | ER Studio                                                       | Database Modeling in ER and IDEF1X                                           |
|                          | DB Artisan                                                      | Database administration and space and security management                    |
| Oracle                   | Developer 2000 and Designer 2000                                | Database modeling, application development                                   |
| Popkin Software          | System Architect 2001                                           | Data modeling, object modeling, process modeling, structured analysis/design |
| Platinum Technology      | Platinum Enterprise Modeling Suite: Erwin, BPWin, Paradigm Plus | Data, process, and business component modeling                               |
| Persistence Inc.         | Pwertier                                                        | Mapping from O-O to relational model                                         |
| Rational                 | Rational Rose                                                   | Modeling in UML and application generation in C++ and JAVA                   |
| Rogue Ware               | RW Metro                                                        | Mapping from O-O to relational model                                         |
| Resolution Ltd.          | Xcase                                                           | Conceptual modeling up to code maintenance                                   |
| Sybase                   | Enterprise Application Suite                                    | Data modeling, business logic modeling                                       |
| Visio                    | Visio Enterprise                                                | Data modeling, design and reengineering Visual Basic and Visual C++          |

# STRUCTURE RELATIONAL DATABASES

Individual task

- **Summarize all the topics that I have presented**

Team Tasks

- Find at least 5 topics about data management using DBMS and describe their data requirement



Barangsiapa yang berangkat menimba ilmu untuk mengamalkan ilmu,  
niscaya ilmu yang sedikit pun akan bermanfaat baginya